Abstract

We critically examine MacroHFT, a hierarchical RL system for minute-by-minute cryptocurrency trading, and partially reproduce its results using the released ETH experts. We study the assumptions of the trading simulator, assess whether the use of RL is justified, and analyse trend and volatility specialisation, dynamic mixing of Q-values, and episodic memory.

The simulator executes trades without latency, slippage, or market impact, which limits the transferability of its results to live trading. Mathematical counterexamples show that a trend and volatility classification does not determine the optimal trading action. Soft mixing of Q-values does not, by itself, guarantee faster adaptation. The authors do not separately test whether volatility specialisation improves on trend-only specialisation, or whether soft mixing improves on a comparable hard router. They also do not measure response times around stress events.

On the ETH test set, two trend experts almost always choose the same action: one stays flat and the other stays long. A fixed expert selected by validation return outperforms every aggregator we study on this test, although the result depends on the selection criterion. Ridge-Static ($V_1 \to V_2$) achieves 13.22% under the authors' normalisation. This is close to the reported 12.73% for MacroHFT without memory, but well below the full model's 39.28%. We work with fixed low-level RL experts and do not reproduce the complete system end to end. Our results do not establish the need for either a regime hierarchy or an additional RL aggregator. Whether RL adds value over simpler trading models remains an open question.

We regard the memory mechanism as the most promising direction for further research. The published ablation shows that it raises return and Sharpe ratio and lowers maximum drawdown on all four assets. It remains to be established whether this improvement comes from making better use of rare events, smoothing Q-values, or reducing risk. It should also be tested against existing alternatives in more realistic trading environments.

1 Trading problem

1.1 Trading environment and available actions

Zong et al. [1] consider trading a single cryptocurrency at fixed one-minute intervals. They formulate the problem as a Markov decision process:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, r, \gamma), \tag{1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{T}$ is the transition function, r is the reward, and $\gamma \in [0, 1)$ is the discount factor.

The agent has two actions: hold no position (*flat*) or hold a fixed-size long position (*long*). Let $P_t \in \{0, 1\}$ denote the position immediately before the decision, and let a_t denote the selected target position:

$$a_t \in \mathcal{A} = \{0, 1\}, \qquad P_{t+1} = a_t. \tag{2}$$

The value 1 means holding m units of the cryptocurrency, while 0 means holding no position. If $a_t \neq P_t$, the simulator executes a fixed-size purchase or sale; otherwise no trade takes place. In other words, only *taker-only* strategies are considered: the position is changed by an aggressive order at the price assumed by the simulator. Limit-order-book snapshots are used as features, but the setting includes no mechanism for executing limit orders.

Let p_t be the closing price of a one-minute interval and let m be the fixed position size. The authors define the reward as the change in portfolio value after transaction costs:

$$r_t = m\Big[a_t(p_{t+1} - p_t) - \delta_t |a_t - P_t|\Big]. \tag{3}$$

Here δ_t is the cost of switching positions, expressed in price units per unit of the asset. With a proportional fee c, it can be written as $\delta_t = c\, p_t$. The experiments use $c = 0.0002$, or 0.02% of the trade value [1]. The simulator charges no fee when the position remains unchanged.

The space of *trading actions themselves* is much narrower here than in optimal execution or market-making problems. Every minute, the agent only has to decide whether to hold a fixed long position or stay in cash. This does not, however, make the optimal policy trivial.

1.2 State and optimisation objective

The observations include OHLCV data and five levels of the limit order book. The authors distinguish features of the current market snapshot from contextual features computed from past observations. At the high level, they add rolling estimates of trend and volatility to describe the longer market context (all features are listed in Table 4 of [1]).

The standard objective of an RL policy is to maximise expected discounted reward:

$$J(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} \gamma^t r_t\right], \tag{4}$$

where $\pi(a \mid s)$ specifies the action distribution in state s. In financial terms, the reward in (3) accumulates trading PnL after fees. The authors also evaluate the results using return, Sharpe ratio, and maximum drawdown.

1.3 Limitations of the execution model

Under this setup, the authors optimise a trading policy π on a historical price path that is unaffected by the policy's actions. When the position switches from flat to long or back, the simulator updates the position, deducts a fee, and continues replaying the same sequence of market observations. It fills the trade at a specified price regardless of order size, order-book conditions, or the time between the signal and execution.

These assumptions need not hold in live trading. We consider latency, liquidity, and the effect of the strategy's own trades on subsequent market behaviour in turn.

Latency and slippage. A trading algorithm makes decisions from information received from the exchange. Some time passes between a market event and the execution of the order sent in response. The data must reach the strategy server, the algorithm must process it, the order must travel to the exchange, and the exchange's trading system must accept and execute it. Let the total latency be

$$\tau = \tau_{\text{data}} + \tau_{\text{calc}} + \tau_{\text{network}} + \tau_{\text{exchange}}. \tag{5}$$

Professional market participants reduce the network component through *colocation*: they place trading servers in or near the data centre that hosts the exchange infrastructure. This both shortens and stabilises the transmission time for data and orders. A signal sent from a remote server crosses additional network hops, making latency longer and less predictable. Colocation does not, however, remove computation time or the exchange's order-processing time.

The available price may change during τ. For example, an algorithm observes a buy signal at the quote p_t^{ask}, but its market order is filled at $p_{t+\tau}^{\text{ask}}$. The difference

$$\varepsilon_t^{\text{lat}} = p_{t+\tau}^{\text{ask}} - p_t^{\text{ask}} \tag{6}$$

measures the buy-side slippage caused by latency. It need not be positive: the price may move in the trader's favour. Signals, however, often arise from events to which other participants also react. A late order may then systematically receive a worse price than the one used when the expected return was assessed.

Latency matters more for strategies that make decisions every second or faster. If a predictable move lasts only a few hundred milliseconds, even a good forecast may lose its practical value before the order is filled. EarnHFT [2] makes trading decisions at one-second intervals but does not include such delays in its execution model.

The one-minute interval used by MacroHFT [1] makes some strategies less sensitive to millisecond-scale latency, but does not eliminate the issue. A decision made once a minute may depend on a fleeting quote change near the end of that minute. Moreover, if an interval's OHLCV values become available only after it ends, execution at that same interval's closing price requires separate justification. Assessing the effect of latency on a particular minute-level policy therefore requires an execution model.

Order-book depth and market impact. Even with zero latency, a market order need not be filled entirely at the best available price.

A limit order book lists the prices and quantities at which other participants are willing to buy or sell the asset. The best ask is the lowest sell price, and the best bid is the highest buy price. Their difference, the *bid–ask spread*, is one component of the cost of immediate execution. A market buyer pays the ask, while a market seller receives the bid.

Only a finite quantity is available at the best price level. Suppose a strategy sends a market order to buy 0.2 ETH when only 0.05 ETH is offered at the best price. The exchange fills the remaining 0.15 ETH at the next, more expensive levels of the book. The average execution price is

$$\bar{p}_t^{\text{buy}}(q) = \frac{1}{q}\sum_{k=1}^{K} p_{t,k}^{\text{ask}} q_{t,k}, \qquad \sum_{k=1}^{K} q_{t,k} = q, \tag{7}$$

where $q_{t,k}$ is the quantity actually filled at level k. If the best quote alone cannot fill the order, the average purchase price exceeds the best ask. An analogous effect occurs on the bid side when selling.

Slippage can therefore arise even without network latency: finite book depth is enough. The larger a market order is relative to available liquidity, the more price levels it may sweep.

MacroHFT constructs features from five-level order-book snapshots, but its reward function uses minute closing prices and fixed transaction costs to determine the execution price. In particular, equation (3) does not account for the cost of sweeping multiple book levels. The closing price need not equal the best bid or ask when the trade is executed.

Sweeping the book is part of the broader issue of *market impact*: the effect of one's own trade on the market price. A large market order consumes available liquidity, may shift the best quotes, and can alter other participants' decisions. Market makers may cancel orders, reduce displayed size, or revise their prices in anticipation of a further move. As liquidity recovers, some of the impact may dissipate while some may persist. The distinction between temporary and permanent impact underlies, among other approaches, the Almgren–Chriss optimal-execution models [6].

The historical simulator does not model this mechanism. The market component of the next state is independent of the strategy's action:

$$\widehat{\mathcal{T}}(s_{t+1}^{\text{mkt}} \mid s_t, a_t) = \widehat{\mathcal{T}}(s_{t+1}^{\text{mkt}} \mid s_t). \tag{8}$$

In a real market, the distribution of future quotes may depend on the direction and size of the strategy's own order, as well as on other participants' response to it. Historical data reveal market dynamics without our intervention, but do not necessarily describe those dynamics after we intervene.

This distinction becomes important as capital grows. In (3), monetary profit scales linearly with the position size m. In reality, increasing m may worsen the execution price more sharply and alter subsequent liquidity, breaking this linear scaling. A strategy that is profitable at small size may lose money when substantially more capital is allocated to it.

For a fixed order size of 0.2 ETH in a liquid market, the impact of a single trade may be small. The simulator's results nevertheless reveal neither the size of that effect nor the capital scale at which the policy retains its edge.

Restriction of the strategy space. Under (2), the agent can change its position only through an aggressive trade. This form of execution reaches the desired position relatively quickly, but requires accepting the market price and paying a taker fee.

A market participant can instead place a limit order to buy at the bid or sell at the ask and wait for an incoming order. It then *provides liquidity* as a maker rather than consuming it as a taker. If a purchase and a subsequent sale are both filled favourably, the participant can earn the bid–ask spread instead of paying it twice. Some exchanges and fee tiers also offer *maker rebates*, payments or discounts for supplying liquidity.

Passive execution is not a cost-free substitute for a market order. A limit order may never be filled: the price can move in the anticipated direction without reaching it. Earlier orders may be ahead in the queue at the same price, so merely touching that price does not guarantee a fill. Finally, a passive order is particularly likely to execute when informed traders are willing to trade against it. This creates *adverse selection*; for example, a bid-side buy may fill just before the price falls further.

High-frequency liquidity-provision strategies must therefore manage queue position, order lifetime and cancellation, non-execution risk, and accumulated inventory. At the same time, for strategies with small expected price moves, savings on the spread and fees can determine whether profit is possible at all. The taker-only restriction excludes such choices, including placing several limit orders at different price levels and working a large order in smaller pieces.

At a fee rate of $c = 0.0002$, a purchase followed by a sale already costs about four basis points of traded value, even before the spread and slippage. If the available predictive signal cannot cover these costs, no profitable active taker policy may exist. Passive execution strategies can remain potentially profitable under the same market conditions. Thus, solving

$$\pi^*_{\text{taker}} = \arg\max_{\pi \in \Pi_{\text{taker}}} J(\pi)$$

does not guarantee finding the best strategy in the broader space of available trading actions Π_{all}.

Transferring results beyond the simulator. These simplifications can change not only the absolute returns of the strategies but also their relative ranking. Let $\widehat{J}(\pi)$ denote the result of policy π in the historical simulator, and let $J_{\text{real}}(\pi)$ denote its expected result under live execution with the same capital.

The inequality

$$\widehat{J}(\pi_A) > \widehat{J}(\pi_B)$$

does not, in general, imply

$$J_{\text{real}}(\pi_A) > J_{\text{real}}(\pi_B).$$

For example, suppose two policies earn 10 and 8 basis points of capital over the same historical period after the fees included in the simulator. The first makes 20 trades and the second makes two.

Suppose live execution imposes an additional loss of one basis point of capital per trade. Then

$$\begin{aligned} \widehat{J}(\pi_A) &= 10, \quad J_{\text{real}}(\pi_A) = -10, \\ \widehat{J}(\pi_B) &= 8, \quad J_{\text{real}}(\pi_B) = 6. \end{aligned} \tag{9}$$

The numbers are illustrative. The example shows that a policy with an advantage in the simulator may trail a more cautious policy once execution is taken into account. At larger order sizes, the difference may be greater still because the additional cost need not remain constant per trade.

A return improvement from adding an RL component establishes its usefulness primarily within the trading environment being modelled. Whether the effect survives latency, limited liquidity, and changes in capital remains a separate empirical question.

Directions for further research. One approach is to enrich the mathematical model of execution. Instead of a fixed fee, one can use a cost function that depends on order size, spread, available book depth, and latency:

$$C_t = C(s_t, q_t, \tau_t), \qquad q_t = m|a_t - P_t|. \tag{10}$$

A richer model can include temporary and permanent price impact, the probability of a partial fill, and liquidity recovery. Its parameters can be estimated from order-book and trade data. This approach makes it possible to study sensitivity to specific execution characteristics at a relatively modest computational cost. Its impact function must, however, be chosen in advance, and other participants' response may be poorly captured by a simple parametric relationship.

A second approach is *agent-based modelling* (ABM). In such an environment, a virtual exchange processes individual orders, cancellations, and trades, while a population of trading agents generates order flow. The policy under study interacts with them through the same execution mechanism, so its actions can change both the book and future observations.

ABIDES [7] is one example of this infrastructure: a discrete-event market simulator that models the order book, participant interactions, and network latency. Events can carry nanosecond-resolution timestamps, allowing the arrival order of messages to be reproduced at scales far below one minute. Nanosecond precision in the clock does not by itself make the participants' behaviour realistic.

The main challenge for ABM is calibration. A useful environment must reproduce not only the return distribution but also order flow, book depth, order lifetimes, the rate of liquidity recovery, and participants' reactions to large trades. Such an environment is much more computationally demanding than replaying historical minute observations. Even a well-calibrated agent-based market remains a model rather than an exact copy of a live exchange.

Both directions permit a more concrete test of the practical value of trading RL architectures: does policy π retain its advantage as latency τ, position size m, and the execution mechanism change? When MacroHFT is compared with simpler policies, such a test may alter the conclusions drawn from a single historical backtest even if the trained models themselves remain unchanged.

2 The MacroHFT architecture: motivation and evaluation

MacroHFT [1] continues a line of research on RL for trading that includes, among others, [3, 4, 5]. Its immediate predecessor is EarnHFT [2]: the system trains a pool of low-level policies, while a separate RL router selects the agent that controls trading. Low-level decisions are made every second and the agent is selected once a minute.

In MacroHFT, both levels operate at one-minute intervals. The authors construct a pool of six agents trained on trend and volatility subsets, and the high-level network mixes their Q-values rather than selecting a single expert. We next examine which problems these changes are meant to solve and which experiments can test the proposed explanations.

2.1 Adding volatility to the market-regime classification

Motivation and architectural choice. Regime specialisation rests on the premise that different trading policies are useful in different parts of the market. MacroHFT refines this idea: classifying regimes by trend alone is insufficient because markets moving in a similar direction may have different volatility. The authors argue that ignoring this distinction weakens both specialisation and risk management [1].

They divide the historical data into blocks of length L. For each block, they compute the slope of a smoothed price series and the volatility of returns. Thresholds determined on the train set define three trend categories and three volatility categories, producing the subsets

$$\begin{aligned} &\mathcal{D}_1^{\text{tr}},\ \mathcal{D}_2^{\text{tr}},\ \mathcal{D}_3^{\text{tr}}, \\ &\mathcal{D}_1^{\text{vol}},\ \mathcal{D}_2^{\text{vol}},\ \mathcal{D}_3^{\text{vol}}. \end{aligned} \tag{11}$$

The authors train a separate dueling DDQN network on each subset. These are two three-category classifications, not the nine intersections of trend and volatility. They set $L = 4320$ minutes for ETH, DOT, and LTC, and $L = 360$ minutes for BTC [1].

What needs to be established. A difference in market statistics does not by itself explain why separate policies should be trained on the corresponding subsets. What matters to a trading decision is the conditional consequence of each action:

$$\Delta^*(s) = Q^*(s, 1) - Q^*(s, 0). \tag{12}$$

With the binary long/flat choice, the sign of $\Delta^*(s)$ determines the action and its magnitude measures the advantage of one action over the other. A trend–volatility partition is useful for specialisation if it makes such differences easier to estimate. The authors do not separately test whether their chosen classification improves policy learning.

A simple counterexample illustrates the problem. Consider two histories of five consecutive prices:

$$\begin{aligned} H_A &= p(1, k, k^2, k, 1), \\ H_B &= p(1, k^{-1}, k^{-2}, k^{-1}, 1), \qquad k > 1. \end{aligned} \tag{13}$$

Both histories end at their initial price p. By symmetry, the slope from regressing price on time is zero in both cases. The simple returns are

$$(k - 1,\ k - 1,\ k^{-1} - 1,\ k^{-1} - 1)$$

for H_A, and the same sequence in reverse order for H_B. The histories therefore have the same mean return and standard deviation.

Their most recent moves point in opposite directions: the price is falling in H_A and rising in H_B. Suppose the conditional dynamics of the next return u_{t+1} exhibit positive autocorrelation:

$$\mathbb{E}[u_{t+1} \mid H_t] = \rho u_t, \qquad \rho > 0. \tag{14}$$

Starting from flat, the agent then remains in cash after H_A but buys after H_B, provided the expected move covers the entry cost. Here, identical linear trend and volatility statistics correspond to different preferred actions.

This example uses a linear trend without preliminary filtering. It demonstrates the information lost by summary statistics: the order

Table 1: Exposure of trend experts on the ETH test set: percentage of minutes spent long. Each expert trades its own sequence of positions.

Expert	Bear	Neutral	Bull
`slope/1`	9.66	5.14	11.04
`slope/2`	26.44	26.71	49.96
`slope/3`	99.88	99.73	99.64

and conditional predictability of moves are not determined by their mean and dispersion. Smoothing or changing the window length is no substitute for testing whether the chosen classification preserves distinctions relevant to trading decisions.

The converse is also possible. Two regimes may differ in return variance but have the same conditional expectation. A one-period policy that maximises expected PnL under identical costs may then choose the same action in both regimes. A difference in volatility alone does not require different neural networks or different policies.

The missing control. Comparing MacroHFT with EarnHFT does not isolate the effect of adding volatility: the expert pool, the hierarchy's timing, the aggregation rule, and the training procedure all change at once. Table 3 of [1] ablates other components, but provides no direct trend-only versus trend-plus-volatility comparison.

Moreover, moving from three experts to six increases the number of trained models. Even a gain from this system would therefore not establish that volatility specialisation itself is useful. A control should use the same number of experts—for example, several independently trained agents for each trend subset, or an alternative partition of comparable size.

Testing the idea requires separating three effects: adding volatility information, changing the training sample, and increasing the number of models. Without such a comparison, the extra axis of specialisation remains an engineering hypothesis whose contribution is unknown.

2.2 The policies produced by separate training

From training regime to expert behaviour. After segmentation, the authors train six policies $\pi_1, \ldots, \pi_6$. Each receives its own subset but uses the same action space $\mathcal{A} = \{0, 1\}$. A regime determines which examples the network sees in training; it does not restrict where the trained network can be applied.

This raises two distinct questions. Does an agent gain a comparative advantage in its assigned regime? And can it recognise when conditions have changed and its previous action is no longer profitable? The first property does not guarantee the second.

The released ETH trend agents make this distinction particularly clear in our reproduction. Table 1 reports the fraction of time spent long within retrospectively labelled Bear, Neutral, and Bull segments of the test period.

The `slope/3` agent holds ETH almost continuously, including during bear segments. By contrast, `slope/1` stays mostly in cash regardless of the trend. The same broad pattern appears on validation: the agents' exposures are 99.56% and 5.84%, respectively. It is not confined to one unusual part of the test period.

Such policies can indeed perform well in their own regimes. An almost constant long position earns money in a rising market, while an almost constant flat position avoids losses in a falling one. This outcome does not require an expert to recognise a regime change; a stable preference for one position is enough.

Our conditional-PnL decomposition is consistent with this interpretation: about 85–87% of the measured contrast in relative specialisation between the two extreme trend agents is associated with the market-exposure component. This is a share of the specialisation measure, not of the agent's total profit.

On the trajectories we study, part of the pool operates mainly as a set of directional long/flat preferences. The high level must recognise when each preference should be trusted. Merely training a bull expert on bull-market data does not create a mechanism for exiting when the market turns bearish.

Does this require separate RL networks? A complex neural network is unnecessary for an almost constant position. Yet a constant action does not prove that the network's numerical Q-values are useless: even if the sign of the long advantage never changes, its magnitude may carry information useful to the hyper-agent.

A targeted ablation is therefore needed: does system performance survive if the extreme experts' recommendations are replaced by constant flat/long policies, or if the magnitude of their Q-advantages is made independent of state? This would separate the value of continuous estimates from the value of position diversity itself.

The other experts' results likewise do not show that partitioning the data requires a hierarchy. `slope/2` achieves 23.87% over the full ETH test set under the authors' return normalisation and outperforms every aggregator we consider. A useful policy trained on one subset can therefore be applied far beyond its original category.

The reason for this result requires a separate test. The subset may have made training easier, the resulting policy may have transferred better to the new period, or the outcome may depend on the particular training run. Testing whether separate training is necessary requires an agent of the same architecture trained on the entire market, together with multiple random initialisations. A single policy with regime features or balanced training examples is a direct alternative to six independent networks.

2.3 Replacing expert selection with weighted Q-values

The proposed solution. In the abstract of MacroHFT, the authors describe a single agent's decisions as "one-sided and highly biased". They associate this property with possible losses in extreme market conditions [1] and propose considering several experts' estimates at once.

Let s_t denote the available state, including the current position. Each network receives the part of this state intended for it. Hard routing takes the form

$$\begin{aligned} i_t &= g(s_t), \\ \pi_{\text{hard}}(s_t) &= \arg\max_{a \in \{0,1\}} Q_{i_t}(s_t, a). \end{aligned} \tag{15}$$

The router g selects one expert, after which the system executes that expert's preferred action.

In MacroHFT, the high-level network produces softmax weights rather than an index:

$$w_i(s_t) = \frac{\exp f_i(s_t)}{\sum_{j=1}^{6} \exp f_j(s_t)}. \tag{16}$$

The aggregate Q-value is

$$Q_{\text{mix}}(s_t, a) = \sum_{i=1}^{6} w_i(s_t) Q_i(s_t, a). \tag{17}$$

With two actions, only the differences

$$d_i(s_t) = Q_i(s_t, 1) - Q_i(s_t, 0), \tag{18}$$

are needed, so the policy becomes

$$\pi_{\text{mix}}(s_t) = \mathbf{1}\left\{ \sum_{i=1}^{6} w_i(s_t) d_i(s_t) > 0 \right\}. \tag{19}$$

The policy chooses flat when the sum is exactly zero.

What actually changes. Equation (19) does not diversify capital across six strategies. Its output is still a single position: either 0 or m units of the asset. Soft mixing changes the rule for selecting that position, but does not create an intermediate exposure.

Moreover, in this binary problem, every action taken by the mixture agrees with the action of at least one expert:

$$\pi_{\text{mix}}(s) \in \{\pi_1(s), \ldots, \pi_6(s)\}. \tag{20}$$

If the weighted sum of $d_i(s)$ is positive, at least one $d_i(s) > 0$. If the sum is non-positive, at least one expert allows the flat action. Given the same information, a sufficiently expressive hard router can select an expert that takes the same action.

This observation does not imply that the two mechanisms are equally easy to learn. It does rule out the explanation that hard routing is inherently unable to produce the required defensive action whereas soft mixing expands the set of available choices. Any advantage must instead come from how the mechanisms learn and use the experts' estimates.

Numerical mixing has a cost of its own. An expert's influence is determined by the product $w_i d_i$, not by its weight alone. For example,

$$w_1 = 0.9, \quad d_1 = 0.1, \qquad w_2 = 0.1, \quad d_2 = -2$$

give a total advantage of -0.11 and hence the flat action. The expert with a 10% weight determines the action in opposition to the expert with a 90% weight.

A Q-advantage estimates a difference in future rewards, not the probability that a recommendation is correct. A large absolute value may reflect either an economically meaningful difference between actions or an estimation error. A plot of weights alone therefore does not show which expert determined the action; at minimum, the products $w_i d_i$ and their sum are needed.

Positive rescaling of a Q-function preserves an individual expert's policy but can change the ensemble's action at fixed weights. The quality and calibration of numerical estimates become a separate requirement, one that a simple vote over the signs of d_i does not have. The authors do not investigate this issue by comparing raw Q-values, their normalised versions, and the policies' own recommendations.

2.4 The claimed faster adaptation is not measured

The original problem. In the introduction to MacroHFT, the authors cite individual policies' delayed response to sudden fluctuations as one reason for changing the architecture. In the results section, they attribute to the new architecture an "ability to adjust quickly to sudden market changes" [1]. Does the proposed system reduce the delay before taking an appropriate trading action after a sharp market change?

Aggregate return over several months cannot answer this question. A policy may identify profitable periods in quiet markets more successfully while responding just as slowly to crashes. The opposite is also possible: earlier exits can reduce crash losses but lower returns through false alarms.

Mixing does not eliminate recognition delay. Both high-level mechanisms can update their decision at the next one-minute step. If the market features do not reveal a change in time, learned weights retain the same problem. Nor is it enough for the pool to contain a suitable expert: that expert's recommendation must exert sufficient influence on the final action.

This is clear even with two experts. Suppose one prefers long and the other flat:

$$d_{\text{long}} = u > 0, \qquad d_{\text{flat}} = -v < 0.$$

Let b_t be the weight of the second expert. Selecting the expert with the largest weight leads to a switch to flat when $b_t > 1/2$. Under Q-value mixing, the exit condition is

$$\begin{aligned} (1 - b_t)u - b_t v &\le 0, \\ b_t &\ge \frac{u}{u + v}. \end{aligned} \tag{21}$$

If $v = u/4$, exiting requires an 80% weight on the flat expert. If $v = 4u$, 20% is enough. For the same path of changing weights, the mixture can therefore switch either later or earlier than hard selection. Faster response does not follow from the mixing operation itself; it depends on the experts' estimates and on the learned rule for using them.

This counterexample does not describe the measured behaviour of MacroHFT. It refutes the general explanation that "several weighted experts respond faster than a single selected expert."

What the paper presents. Figure 4 contains individual examples of trading decisions, including an exit during a fall in LTC. For the same event, however, the authors do not show EarnHFT's response, analyse a systematically selected sample of stress episodes, or report a distribution of exit delays.

Figure 5 shows expert weights averaged over 60 minutes. Such a plot cannot reveal whether the weights changed before a sharp move, one minute afterwards, or much later within the same hour. Moreover, weights without Q-advantages do not determine when the trading position changed.

Even the aggregate risk metrics show no unconditional advantage for the new system. In Table 1, maximum drawdown on DOT is 15.89% for MacroHFT versus 5.80% for EarnHFT; on LTC it is 14.24% versus 9.63% [1]. These figures do not measure adaptation delay, and they cannot support a claim that the new architecture systematically provides better protection from large losses.

The authors do not test the claimed link between the architecture and rapid adaptation. They report final trading results and behavioural illustrations, but do not measure the advantage used to motivate the change in policy-selection mechanism.

An experiment that would test the claim. The benchmark set already contains a hard-routing baseline: EarnHFT is one of the algorithms being compared. The missing control is a learned hard router and a learned soft mixer operating on the same pool of fixed experts, with identical inputs and trading frequency and comparable training.

Such an experiment should identify stress events using a rule fixed in advance. It should then measure the delay before a position change, losses in the event window, and the rate of false defensive exits outside those windows. The initial position and the opportunity to reduce risk must be handled explicitly: an agent that was already in cash before an event did not make a fast exit.

An additional control would freeze the mixture weights before an event while allowing the experts' Q-values to change. Comparing this version with dynamic weights would show whether reweighting itself helps, or whether the low-level networks provide the defensive action. The system's total PnL cannot separate these mechanisms.

2.5 What simple aggregation alternatives show

The substitution being tested. In our experiments, we hold fixed the six released ETH experts, the authors' features, the position size, and the fee. We change only the rule that turns the experts' estimates into a trading action. Each strategy passes its own previous position to the experts.

In addition to a uniform mixture, majority vote, and normalised Q-advantages, we test a single fixed expert, a tabular hard router,

and a supervised Ridge controller. The tabular router is trained on the nine combinations of trend and volatility levels and selects one of the six experts. It is a simple learned alternative, not a reproduction of EARNHFT's RL router.

Nor can Ridge be described merely as replacing softmax weights with fixed coefficients. In the notebook, Ridge predicts the next one-minute return:

$$\widehat{\mu}_t = b + \sum_{i=1}^{6} \beta_i \frac{d_i(s_t)}{\sigma_i}, \tag{22}$$

where the scales σ_i are estimated on the meta-model's training segment. We then apply a fee-aware decision rule:

$$\pi_{\text{Ridge}}(s_t) = \arg\max_{a \in \{0,1\}} \{a\widehat{\mu}_t - c|a - P_t|\}. \tag{23}$$

From flat, the rule enters when $\widehat{\mu}_t > c$; from long, it exits when $\widehat{\mu}_t \leq -c$. This is a distinct and simple way to manage a position using the outputs of RL experts.

We train the main Ridge variant on the first half of validation (V_1) and select its regularisation on the second half (V_2). The contextual version adds interactions between the expert signals and rolling trend and volatility. The low-level policies remain unchanged in both cases.

A result that needs explaining. On this test, the best variant we consider uses the `slope/2` expert selected by validation return and never switches experts at all. Its MTM Return is 23.28%, compared with 9.08% for Majority. Exposure is slightly lower (32.09% versus 35.38%), while maximum drawdown is almost identical (9.54% versus 9.65%). The gap cannot be explained simply by spending more time long.

At the same time, the result depends on the expert-selection procedure. The original selection by validation Sharpe chose `vol/2` and produced a negative test return. The observation therefore establishes a narrower fact, not the general reliability of fixed selection: among the released policies, one expert performs better in this experiment than all of our combinations. Such a baseline is necessary when evaluating the incremental value of aggregation.

Static Ridge achieves 13.22% under the authors' normalisation. In Table 3, the authors report 12.73% for MACROHFT without memory and 39.28% for the full model [1]. A simple supervised alternative thus reaches a result numerically comparable to the published no-memory configuration. This is a reason to include it in a direct comparison, rather than attribute the full system's entire result to a learned voting mechanism.

The remaining tests do not support the idea that any simplification will do. Normalising the Q-advantages worsens the result. The tabular hard router barely improves on Majority in return but substantially increases drawdown and exposure. Contextual Ridge trails Static Ridge in return. Our evidence supports neither a universal advantage for hard routing nor the claim that mis-scaled Q-values explain every problem.

2.6 Episodic memory in hyper-agent training

2.6.1 Motivation

After training specialised experts and introducing a hyper-agent, the stability of the latter's training remains a concern. In Section 4.3 of MACROHFT [1], the authors identify two causes. The first is high reward variance among similar market states. The second is rare, sharp fluctuations that standard experience replay is presumed not to use effectively enough.

Experience replay already stores past transitions and reuses them during training. Uniform sampling, however, reproduces the frequencies in the observations: ordinary states appear in training batches far more often than rare ones. If a few episodes of large losses determine the quality of the entire strategy, this allocation of updates may be undesirable.

The authors propose augmenting training with episodic memory. For the current state, the method retrieves similar transitions and uses their stored value estimates as an additional training signal. The expectation is that this will align the Q-values of nearby states and help produce a policy that responds more quickly to sharp market changes.

This approach extends an existing line of research. Episodic Memory Deep Q-Networks already used memory to provide additional training for a Q-network [8], while Neural Episodic Control combined a learned state representation with a table of value estimates [9]. MACROHFT cites both studies. The research question is which problem of the trading hyper-agent is solved by this particular memory design, and what explains its advantage.

2.6.2 Memory design and operation

Let $Q_\theta(s, a)$ denote the hyper-agent's Q-function, where θ are the parameters of the trainable high-level network. As before, this network constructs its Q-function by weighting the low-level experts' estimates. The memory is stored separately from the network parameters:

$$\mathcal{M}_{\text{mem}} = (K, E, V). \tag{24}$$

K contains retrieval keys, E contains states and actions, and V contains stored action-value estimates.

Adding a transition. For an observed transition (s_j, a_j, r_j, s'_j), the encoder computes the key

$$h_j = \psi_{\theta_j}(s_j), \tag{25}$$

where ψ_{θ_j} is the hyper-agent's state encoder at the time the entry is created. For a non-terminal transition, the stored value is

$$v_j = r_j + \gamma \max_{a'} Q_{\theta_j}(s'_j, a'). \tag{26}$$

In a terminal state, the code sets the future reward to zero.

v_j is not the observed profit from the rest of the trade or the completed episode. Only r_j is observed directly; the network computes the remainder from its own Q-value. This is a one-step bootstrapped target stored for later training.

The table has finite capacity. When it is full, the code removes the oldest entry under a FIFO rule. The memory thus retains recent transitions rather than a permanent archive of the most dangerous market events.

Retrieving similar states. For state s, the code computes the current key $h = \psi_\theta(s)$ and then selects the k stored keys closest in Euclidean distance. Let $\mathcal{N}_k(h)$ be the set of their indices.

The published implementation uses the kernel

$$\kappa(h, h_j) = \frac{1}{\|h - h_j\|_2^2 + \varepsilon}, \qquad \varepsilon > 0. \tag{27}$$

Closer entries receive more weight. Among the retrieved neighbours, the implementation then uses only entries with the same action:

$$\omega_j(s, a) = \frac{\kappa(h, h_j)\mathbf{1}\{a_j = a\}}{\sum_{\ell \in \mathcal{N}_k(h)} \kappa(h, h_\ell)\mathbf{1}\{a_\ell = a\}}. \tag{28}$$

The auxiliary estimate is

$$Q_M(s, a) = \sum_{j \in \mathcal{N}_k(h)} \omega_j(s, a) v_j. \tag{29}$$

Table 2: Selected results for alternative ways of using the six ETH experts on the test set from June to October 2023. MTM Return, Sharpe, and MDD use a common initial capital; Author-style Return uses `required_money` as its denominator. Exposure is the percentage of minutes spent long.

Method	MTM, %	Author-style, %	Sharpe	MDD, %	Exposure, %
Uniform Q	5.72	5.14	0.773	12.92	29.64
Normalized Q	-4.04	-3.53	-0.333	18.07	34.31
Majority	9.08	9.31	1.128	9.65	35.38
Tabular hard router	10.04	9.94	0.915	18.17	82.93
Ridge-Static, $V_1 \to V_2$	13.36	13.22	1.414	12.96	46.80
Ridge-Contextual, $V_1 \to V_2$	6.88	7.22	0.992	9.37	24.67
Fixed expert: selected by validation return	23.28	23.87	2.424	9.54	32.09
Fixed expert: selected by validation Sharpe	-22.47	-18.74	-3.116	25.28	22.80

Equations (27)–(28) match the published code [12]. Equation (7) in the paper uses the distance itself rather than its inverse. This difference changes the neighbours' relative contributions; we use the implemented version here. Until the table is full, or if none of the retrieved neighbours has the relevant action, the code substitutes the ordinary one-step estimate for Q_M.

Policy training and execution. The loss function includes the retrieved target:

$$\begin{aligned} \mathcal{L}(\theta) &= \mathcal{L}_0(\theta) + \lambda_M \mathcal{L}_M(\theta), \\ \mathcal{L}_M(\theta) &= \mathbb{E}\left[\big(Q_\theta(s,a) - Q_M(s,a)\big)^2\right]. \end{aligned} \tag{30}$$

Here $\lambda_M \geq 0$ controls the strength of the auxiliary training, while $\mathcal{L}_0$ collects the original method's other terms, including the TD error and an auxiliary DP target. The implementation computes Q_M when collecting a transition and stores it with that transition in the ordinary replay buffer.

During test-time trading, the model performs no memory lookup: the trained expert and hyper-agent networks determine the action [12]. Memory affects the outcome through the parameters learned during training. The explanation that "the agent saw a crash, recalled a similar case, and exited the position" therefore does not describe this model's execution.

The extra training may still improve responses by producing a more suitable policy. But the speed at which transitions are learned during training and the speed at which the policy exits a position in a new market are different properties. The former does not establish the latter.

2.6.3 Published results

The authors test the value of memory with the *w/o-MEM* ablation. For memory, they directly compare the full system with a version lacking that component; no equivalent comparison is provided for volatility specialisation or soft aggregation.

Table 3 reproduces the results from Table 3 of [1]. We did not retrain the hyper-agent as part of this review; the table contains the published results, not our own memory experiment.

With memory, return and Sharpe ratio rise and MDD falls on all four assets. The return difference is 26.55 percentage points on ETH and 26.82 on LTC. A general objection to architectural complexity does not negate this positive result.

The table does not, however, separate the possible causes of the improvement. It provides no distribution of the effect across repeated training runs, and the four assets' test periods overlap substantially in calendar time. It therefore does not reveal whether the gain is robust to random initialisation or to a change in market period.

In discussing Figure 6, the authors attribute the difference between the models to the no-memory variant's late response to a fall in ETH. This illustrates the behaviour of two trained policies. It cannot replace a comparison on a systematically selected sample of stress events, nor does it show how much of the total improvement specifically comes from avoiding rare losses.

2.6.4 Which problem does the mechanism solve?

The motivation for memory combines several possible shortcomings. Rare events may receive too few training updates, while Q-values may be unstable. In addition, an objective that maximises mean reward may permit unacceptable risk. These problems require different solutions. A "with-memory versus without-memory" ablation cannot distinguish between the corresponding explanations of the effect.

Insufficient use of rare transitions. If the problem is the distribution of training examples, their sampling frequency is what needs to change. Prioritized Experience Replay raises the probability of resampling transitions with a large TD error and uses importance-sampling corrections to control bias [10]. In a financial setting, one can also test event-based sampling: identify sharp moves or large losses in the train set in advance and ensure their representation in training batches.

PER does not guarantee special attention to financial disasters in particular: an ordinary noisy transition may also have a large TD error. It does, however, directly test the hypothesis that standard training allocates updates poorly across observations.

In MacroHFT, neither storing an entry nor retrieving it later depends directly on the size of a loss or the rarity of the market event. FIFO favours recent observations, while retrieval favours nearby states. A rare episode may be evicted from the table; while it remains there, its contribution depends on how often it qualifies as a relevant neighbour.

This does not mean that all information about the episode disappears: it may remain in the network parameters and the ordinary replay buffer. But the memory design provides no dedicated mechanism for retaining crises over the long term or prioritising them.

Moreover, increasing the share of crisis examples without correcting their weights changes the distribution against which the model is trained. If the purpose is to induce more cautious behaviour, this change should be stated as part of the training objective, not merely as a remedy for too few observations.

Stabilising and smoothing Q-values. The actual loss in (30) admits another explanation: memory creates a second target by averaging stored estimates. This may reduce the variance of training targets and alter the optimisation path even in ordinary, non-extreme states.

Algebraically, consider one update with fixed TD target y and memory value $z = Q_M(s,a)$. The isolated quadratic part of the loss is

$$\ell(q) = (q-y)^2 + \lambda_M (q-z)^2, \qquad q = Q_\theta(s,a). \tag{31}$$

Table 3: Memory ablation in MacroHFT, from Table 3 of [1]. TR is return, ASR is the annualised Sharpe ratio, and MDD is maximum drawdown. All values are taken from the paper.

	Without memory			Full MacroHFT		
Market	TR, %	ASR	MDD, %	TR, %	ASR	MDD, %
BTC	2.03	0.49	6.49	3.03	0.61	5.41
ETH	12.73	1.26	20.87	39.28	3.89	9.67
DOT	2.41	0.34	27.23	13.79	0.97	15.89
LTC	-8.66	-0.58	22.03	18.16	1.50	14.24

It is minimised at

$$q_{\text{eff}} = \frac{y + \lambda_M z}{1 + \lambda_M}. \tag{32}$$

Thus, the extra term shifts the target estimate towards the stored values. This is a property of the two quadratic terms, not a solution of the full network-training problem.

Such smoothing is useful if neighbouring entries provide additional information about the same conditional value. If nearby embeddings instead join states in which actions have different consequences, averaging may obscure precisely the distinction that matters. This concern is especially acute at a regime boundary, where the policy is expected to change its decision quickly.

The initial observation that "similar states have very different rewards" does not identify the source of the dispersion. It may reflect random noise, estimation error, or missing information in the input state. Averaging may help in the first case; in the last, it cannot recover the missing feature needed to distinguish the situations.

The stored values v_j also contain the hyper-agent's own predictions. Memory provides no independent check on its long-horizon estimates. It may smooth errors, but it may also propagate a systematic error across similar states. Bootstrapped estimates are not themselves a defect: ordinary Q-learning uses them too. Their presence does, however, limit the interpretation of the table as a store of verified correct decisions.

Natural competitors for this hypothesis include other auxiliary Q-targets, value averaging, and regularisation. A comparison with them would reveal whether neighbour retrieval and a separate table are necessary, or whether the main effect comes from an additional stabilising term.

The policy's attitude to risk. Even perfect recall of every crisis transition does not compel an agent to avoid a risky strategy. If the strategy has positive expected reward, maximising the expectation may favour it over a safer alternative.

The MacroHFT loss includes an error that aligns Q-values, but no explicit constraint on tail losses. The lower drawdown in the table is therefore an empirical training result, not a direct consequence of an optimised risk criterion.

If the goal is to limit rare, large losses, it can be stated explicitly. For example, for total reward G_π, consider

$$J_{\text{risk}}(\pi) = \mathbb{E}[G_\pi] - \lambda_R \,\text{CVaR}_\alpha(-G_\pi), \tag{33}$$

where $\lambda_R \geq 0$, and CVaR_α measures the mean loss in the upper tail of the loss distribution; at $\alpha = 0.95$, this is the worst 5% of outcomes. Risk-sensitive formulations using CVaR predate MacroHFT [11].

Equation (33) defines an alternative problem; it does not describe a hidden effect of memory. It requires comparison along the return–risk trade-off. Memory may improve reward estimation, while a risk-sensitive objective changes the policy's preferences; the two mechanisms can complement one another.

2.6.5 What should be tested next

The first step is to decide which explanation is being tested. Resampling data, smoothing Q-targets, and changing the attitude to risk are not interchangeable ablations.

To test the rare-event explanation, ordinary replay, PER, event-based sampling, and MacroHFT's memory should be compared with the same low-level experts and hyper-agent. In addition to final PnL, the experiment should measure how often crisis transitions take part in training, which entries memory retrieves, and whether losses fall in new stress episodes. Without this evidence, the rare-event explanation remains a conjecture.

Testing the role of retrieval requires a control with the same auxiliary loss term but a different construction of Q_M. Nearest states can be compared with random entries of the same action and similar age, as well as with uniform averaging over the retrieved neighbours. This separates the selection of relevant transitions from the mere presence of a second training target. Q-value quality should also be evaluated on held-out data: reducing error against the model's own bootstrapped target does not yet establish a better reward prediction.

If the effect is attributed to risk management, the model should be compared with an explicit risk objective. Tail losses, drawdowns, and foregone returns should be evaluated instead of choosing a winner by total profit alone. Differences in market exposure should also be reported: reducing losses simply by being in the market less often is not equivalent to better recognition of dangerous states.

All comparisons should use the same training data, the same hyper-agent architecture, and comparable budgets for tuning and updates. Training time and the cost of memory retrieval should be reported separately. Repeated initialisations and non-overlapping test periods would help distinguish a robust effect from the particulars of the selected trained policies.

The published ablation supports the usefulness of the added module in this particular system, but does not establish which training defect it remedies. Justifying memory as a mechanism for handling extreme events requires precisely this separation of mechanisms. Otherwise, the high return confirms a successful combination of components, but does not explain why episodic memory should be preferred to existing ways of using experience and managing risk in the same task.

References

[1] C. Zong, C. Wang, M. Qin, L. Feng, X. Wang, and B. An. MacroHFT: Memory Augmented Context-aware Reinforcement Learning On High Frequency Trading. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 4712–4721. DOI: 10.1145/3637528.3672064.

[2] M. Qin, S. Sun, W. Zhang, H. Xia, X. Wang, and B. An. EarnHFT: Efficient Hierarchical Reinforcement Learning for High Frequency Trading. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(13), 2024, pp. 14669–14676. DOI: 10.1609/aaai.v38i13.29384.

[3] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai. Deep Direct Re-

inforcement Learning for Financial Signal Representation and Trading. *IEEE Transactions on Neural Networks and Learning Systems*, 28(3), 2017, pp. 653–664 (published online in 2016). DOI: 10.1109/TNNLS.2016.2522401.

[4] T. Théate and D. Ernst. An Application of Deep Reinforcement Learning to Algorithmic Trading. *Expert Systems with Applications*, 173, 2021, article 114632. DOI: 10.1016/j.eswa.2021.114632.

[5] T. Zhu and W. Zhu. Quantitative Trading through Random Perturbation Q-Network with Nonlinear Transaction Costs. *Stats*, 5(2), 2022, pp. 546–560. DOI: 10.3390/stats5020033.

[6] R. Almgren and N. Chriss. Optimal Execution of Portfolio Transactions. *The Journal of Risk*, 3(2), 2001, pp. 5–39. DOI: 10.21314/JOR.2001.041.

[7] D. Byrd, M. Hybinette, and T. H. Balch. ABIDES: Towards High-Fidelity Multi-Agent Market Simulation. *Proceedings of the ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, 2020, pp. 11–22. DOI: 10.1145/3384441.3395986.

[8] Z. Lin, T. Zhao, G. Yang, and L. Zhang. Episodic Memory Deep Q-Networks. arXiv:1805.07603, 2018. https://arxiv.org/abs/1805.07603.

[9] A. Pritzel, B. Uria, S. Srinivasan, A. Puigdomènech Badia, O. Vinyals, D. Hassabis, D. Wierstra, and C. Blundell. Neural Episodic Control. *Proceedings of the 34th International Conference on Machine Learning*, PMLR 70, 2017, pp. 2827–2836. https://proceedings.mlr.press/v70/pritzel17a.html.

[10] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized Experience Replay. *International Conference on Learning Representations*, 2016. https://arxiv.org/abs/1511.05952.

[11] Y. Chow, A. Tamar, S. Mannor, and M. Pavone. Risk-Sensitive and Robust Decision-Making: a CVaR Optimization Approach. *Advances in Neural Information Processing Systems*, 28, 2015, pp. 1522–1530. https://arxiv.org/abs/1506.02188.

[12] ZONG0004. MacroHFT: official implementation. Version reviewed: commit `31e5ef41f93b2aea6e63e6ae2267675c997a8e54`. Files `RL/util/memory.py` and `RL/agent/high_level.py`. https://github.com/ZONG0004/MacroHFT/tree/31e5ef41f93b2aea6e63e6ae2267675c997a8e54.